entelect HACKATHONS

presents

UNIVERSITY CUP

AGE OF ENTELAND

Age of Enteland

Introduction

Welcome, Mayor! The region's towns have fallen on quiet times, and the council has appointed you to change that. Your task is to create the optimal economic strategy given information about the map layout, the towns, the resource nodes, and the recipes available to you. Your strategy must be resource-efficient and well optimised for time to maximise points.

The currency of the region is **Enteloot**.

Goal

As the mayor, you must develop a program that generates the actions your character will take during the run, namely:

- Where to travel, and which routes to take.
- When to gather resources at nodes versus buying them at towns.
- What to craft, and where.
- Where to sell resources and crafted goods.
- Which upgrades to build, in which towns, and in what order.

You will need to consider each town's prices, production, and affinities to optimise where you gather, craft, sell, and build. Time and Enteloot pull against each other throughout: almost everything you gain in one costs you the other.

You will ultimately be rewarded for growing the towns' economies as much as possible within the tick limit. Infrastructure is the primary driver of score, and spreading development across towns earns a multiplier. Hoarded Enteloot scores far less than Enteloot invested.

Assumptions

The following assumptions apply to the simulation:

1. **Global Tick Clock**
   The simulation runs on a single global clock measured in ticks. The run lasts a total number of ticks defined in the JSON level file. If an action's tick cost would push the clock past `total_ticks`, that action does not execute: it is skipped, logged, and the clock is advanced to `total_ticks` (crediting passive systems for the remaining window) so the run ends cleanly. This cutoff applies uniformly to every action type - travel, gather, buy, sell, craft, build, and upkeep - not only to movement.
2. **Sequential Action Execution**
   Player actions execute strictly in submission order. Each action consumes its full tick cost before the next action begins.

3. **Validation at Execution Time**
   Each action is validated when it is reached, not when it is submitted. An action is invalid if its prerequisites (location, resources, components, Enteloot) are not met at that moment.
4. **Invalid Action Handling**
   An invalid action - whether from an unmet prerequisite or a malformed entry (missing field, wrong type, unknown `type`) - is skipped, consumes 1 tick, and is logged. The run does not halt. Only a submission that fails to parse entirely is handled differently (see Submissions).
5. **Passive Systems Fire on the Clock**
   Town production, town Enteloot generation, and active boost timers advance with the global clock, regardless of where the player is or what they are doing.
6. **Auto storage of generated resources**
   Accumulated production and Enteloot by a town are automatically assigned and accessible to the player.
7. **Determinism**
   There is no randomness anywhere in the simulation. Route tolls are fixed costs, not probabilities. Given the same input, the same actions always produce the same result.
8. **Starting State**
   The player begins at the starting town defined in the JSON level file, with the starting Enteloot amount defined in the JSON level file, an empty inventory, and all recipes unlocked for that level.
9. **No Debt**
   The player's Enteloot can never go below zero. Any action that would require more Enteloot than the player holds is invalid.
10. **Unlimited Inventory**
    There is no carry limit on resources, components, or crafted goods.
11. **Fixed Prices**
    Buy prices, sell prices, and item rates are fixed for the duration of a run.
12. **Atomic Travel**
    Travelling consumes the full edge weight in ticks and the player arrives at the destination at the end. The player cannot act mid-route.
13. **Realism**
    The map, towns, resources, and prices are fictional and are not necessarily realistic.

Constraints

The Map

The map is a graph. Vertices are either **towns** or **resource nodes**. Edges are routes weighted by travel time in ticks.

- Travelling is done between two connected vertices by specifying the destination.
- Some pairs of vertices are connected by both a standard route and a fast route. Fast routes have a lower tick weight but charge a fixed Enteloot toll on entry.

- When both exist, the travel action's optional `fast` field picks which one: omitted or `false` takes the standard route, `true` takes the fast route. Requesting `fast: true` where no fast route exists between that pair is an invalid action.
- If the player cannot pay the toll, the travel action is invalid.
- The map is not required to be fully connected. Each node is connected only to some of the other nodes as defined by the routes in the level file.
- Players may revisit towns and resource nodes any number of times.
- Resource nodes are not depleted by gathering and may be gathered from repeatedly.
- Fast-route tolls are charged every time the player traverses that route.

Travel time formula:

```
travel_time = edge_weight_ticks
travel_cost = edge_toll_enteloot   (0 for standard routes)
```

Example: Demacia to Piltover has a standard route of weight 5 with no toll, and a fast route of weight 2 with a toll of 50 Enteloot. Taking the fast route saves 3 ticks for 50 Enteloot.

Towns

Towns produce resources and Enteloot passively.

Trickle formula (applies to both resources and Enteloot):

```
cycles_completed = floor(current_tick / rate)
accumulated = cycles_completed × amount
```

If Production or Civic upgrades modify `amount` (doubling, or a percentage buff), the modified `amount` is floored to the nearest integer before use in this formula.

Example: Demacia produces 2 wheat and 2 sheep every 10 ticks. At tick 47, four cycles have completed. The player has accumulated 8 wheat and 8 sheep.

Each town also has:

- **Affinities:** A crafting affinity reduces craft time at that town from 2 ticks per item to 1 tick per item, for goods and construction components alike.
- **Item-rates:** The prices the town pays for crafted goods. Towns pay the most for goods made from resources they do not produce.
- **Buy availability:** Each town sells the raw resources it produces at the buy prices in the Resources table. Buying is unlimited and is not constrained by accumulated production.
- **Upkeep (Level 4 only):** A boost action that doubles the town's Enteloot production for 50 ticks (see Activities). Triggering it again while already active refreshes the duration rather than stacking the multiplier.

Resource Nodes

Resource nodes are separate vertices where gathering activities are performed. Gathering returns far more per tick than town trickle, but costs travel time to reach.

Node type	Resource	Gathering activity	Gather time
Fields	Wheat	Harvesting	2 ticks
Forest	Wood	Logging	2 ticks
Quarry	Stone	Quarrying	2 ticks
Clay pit	Clay	Digging	2 ticks
Fishing grounds	Fish	Fishing	2 ticks
Pasture	Sheep	Farming	2 ticks
Mine	Ore	Mining	3 ticks

Mine nodes only appear on Level 3+ maps.

Each individual node defines its own yield in the JSON level file. Performing the gathering activity at a node adds `yield` units of that node's resource to the player's inventory.

Resource nodes are permanent and may be gathered from any number of times during a run.

Example: Node N1 is a Fields node with yield 6. One Harvesting action at N1 costs 2 ticks and adds 6 wheat to inventory.

Resources

There are seven resources. Resources have no subtypes; they are differentiated by quantity, price, and where they come from.

Resource	Sell price	Buy price
Wheat	2	4
Wood	3	5
Stone	3	5
Clay	4	6
Fish	4	6
Sheep	5	8
Ore	6	Cannot be bought

For every resource there are three ways to obtain it:

1. Wait for town trickle (free, slow).
2. Travel and gather at a node (costs time).
3. Buy at a town that produces it (costs Enteloot, instant).

Buying is always faster but always worse per Enteloot. Purchases are unlimited and are not constrained by a town's accumulated production.

Ore is the exception: no town produces or sells it. It can only be gathered at Mine nodes, which appear from Level 3.

Recipes

Recipes convert resources into sellable goods. All recipes are available from the start of the level in which crafting is enabled. Crafting takes 2 ticks per item, reduced to 1 tick per item at a town with crafting affinity. A craft action with quantity n consumes n x craft time in ticks and n sets of inputs.

Recipe	Inputs	Craft time	Sell range across towns
Bread	3 wheat	2 ticks	10 to 40
Fish-n-chips	2 fish + 1 wheat	2 ticks	10 to 50
Stew	1 sheep + 1 fish + 1 wheat	2 ticks	25 to 50
Wooden-crafts	4 wood	2 ticks	10 to 50
Furniture	3 wood + 1 sheep	2 ticks	35 to 65
Stone-works	5 stone	2 ticks	20 to 60
Roof-tiles	3 clay + 2 stone	2 ticks	40 to 70
Wool-garments	3 sheep	2 ticks	30 to 60
Pottery	4 clay + 1 wood	2 ticks	50 to 70

The exact rate each town pays for each good is defined in that town's item-rates in the JSON level file.

Every town defines an item-rate for every crafted good, therefore crafted goods may be sold at any town.

Construction Components

Construction components are recipes that are never sold, only consumed by Building actions. Some components consume other components, so construction has its own production chain. Component crafting follows the same timing rule as goods: 2 ticks per item, 1 tick per item at a town with crafting affinity, which makes affinity towns the natural construction workshops.

Component	Inputs	Craft time
Planks	2 wood	2 ticks
Thatch	2 wheat	2 ticks
Stone-blocks	3 stone	2 ticks
Mortar	1 clay + 1 stone	2 ticks
Bricks	2 clay + 1 mortar	2 ticks
Rope	2 sheep	2 ticks
Fencing	2 wood + 1 rope	2 ticks
Kiln-glass	2 clay + 2 wood	2 ticks
Nets	1 rope + 1 fencing	2 ticks
Iron-fittings	2 ore + 1 wood	2 ticks

Iron-fittings can only be crafted from Level 3, when Mine nodes appear. It is required for the police-station and for tools.

Upgrades

Every upgrade is built from components plus Enteloot. Building consumes the listed components and Enteloot from the player and permanently adds the upgrade to the town where the Building action is performed. Each upgrade can be built once per town.

Production upgrades double the town's trickle amount for the matching resource:

Upgrade	Boosts	Components	Enteloot	Build time	Prerequisite	Score value
Farmhouse	Sheep	3 planks + 2 thatch	500	3 ticks	None	1000
Pier	Fish	4 planks + 2 nets	600	3 ticks	None	1000
Fertilised-fields	Wheat	2 fencing + 2 thatch	500	3 ticks	None	1000
Quarry	Stone	3 stone-blocks + 2 planks	600	3 ticks	None	1000
Woodlands	Wood	2 fencing + 2 rope	500	3 ticks	None	1000
Pottery-house	Clay	4 bricks + 2 planks	700	3 ticks	None	1000

Production upgrade prerequisites are evaluated per town. A production upgrade built in one town does not satisfy prerequisites for civic upgrades in another town.

Civic upgrades buff town Enteloot generation. Percentage bonuses stack additively with one another. Prerequisites are per town:

Upgrade	Effect	Components	Enteloot	Build time	Prerequisite	Score value
Rec-center	Enteloot amount +20%	4 planks + 3 bricks + 1 rope	1200	4 ticks	Any 1 production upgrade	3000
Fire-station	Boost duration +50% (50 → 75 ticks)	5 bricks + 3 stone-blocks + 2 rope	1800	4 ticks	Any 2 production upgrades	4000
School	Enteloot amount +50%	6 bricks + 3 planks + 2 kiln-glass	2000	5 ticks	Rec-center	5000
Police-station	Enteloot rate −2 ticks (min 1)	6 bricks + 4 stone-blocks + 2 iron-fittings	2200	5 ticks	Fire-station	5000

Library	Enteloot amount +50%	5 bricks + 5 planks + 2 kiln-glass	2500	5 ticks	School	6000

All civic upgrade prerequisites are evaluated per town. For example, a Rec-center requiring one production upgrade, or a Fire-station requiring two production upgrades, must satisfy those prerequisites in the same town where the civic upgrade is being built.

If multiple Enteloot amount bonuses are active on a town (for example Rec-center, School, and Library), their percentage bonuses are added together before being applied. If a production upgrade or upkeep effect also modifies Enteloot generation, those effects are applied in addition to the combined percentage bonus.

The police-station requires iron-fittings and is therefore only buildable from Level 3.

Tools (Level 3+)

Tools are craftable items that act as permanent buffs for the rest of the run. They are never sold, are not consumed, and each tool can be crafted at most once per run. Tool crafting follows the standard craft timing rule (2 ticks, 1 with crafting affinity).

Tool	Inputs	Effect
Boots	2 iron-fittings + 2 rope	All travel times reduced by 1 tick (minimum 1) per edge
Pickaxe	2 iron-fittings + 2 planks	All gather times reduced by 1 tick (minimum 1)

Because both tools require iron-fittings, acquiring them means a trip to a Mine node first. Tools change the effective weights of the map, so routes worth planning before boots may not be the routes worth planning after.

Activities

All activities consume ticks and can only be performed if all prerequisites are met at execution time.

Activity	Where	Ticks	Description
Travel	Between connected vertices	Edge weight	Move to a connected town or node. Fast routes charge a toll.
Buy	At any town	1	Buy resources the town produces, at buy price.
Sell	At any town	1	Sell raw resources at their global sell price, or crafted goods at the current town's item-rate. Every town supports selling every crafted good.
Craft	At any town	2 per item (1 with crafting affinity)	Convert resources into recipe items or components. Quantity n costs n × craft time and n sets of inputs.
Build	At any town	Per upgrade	Consume components and Enteloot to construct an upgrade at this town.

Gather	At a resource node	Node gather-time (2, or 3 at mines)	Perform the node's gathering activity and receive its yield.
Upkeep	At any town	5	Doubles the current town's Enteloot production for 50 ticks. Triggering again while active refreshes the duration rather than stacking the multiplier.

Levels

Each level file defines the characteristics of the run, including the map, towns, nodes, recipes, upgrades, and run parameters. Each level adds new factors while building on the previous ones (i.e. Level 2 contains the rules from Level 1, in addition to its own).

Data organisation: Resources, Recipes, Construction Components, Upgrades, and Tools (the tables under Constraints) are global constants: their definitions are identical across every level and are not part of the level JSON file. A level file varies only `run` parameters, `towns`, `nodes`, and `routes` (see the JSON level file example); which mechanics are enabled (crafting, building, fast routes, mines, tools, upkeep) is determined by the level number itself, not by per-level data.

Level 1

Basic rules to help you get familiar with the problem. Crafting and Building are disabled. Your focus will be on the following:

- Navigating the map and reading edge weights.
- Choosing between waiting for town trickle, travelling to gather at nodes, and buying outright.
- Selling resources at the towns that produce them least.
- Managing the tick budget across travel, gathering, and trading.

Level 2

Crafting and Building are introduced. In addition to Level 1, focus on:

- Crafting resources into goods and hauling them to the towns that pay most.
- Crafting construction components and their dependency chains.
- Building production upgrades early enough that their boosted trickle pays back.
- Sequencing civic upgrades through their prerequisites to drive score.

Level 3

Fast routes, Mine nodes, and tools are introduced. In addition to previous levels, focus on:

- Deciding when a toll is worth the ticks it saves.
- Gathering ore at Mine nodes, the only resource that cannot be bought.
- Crafting iron-fittings to unlock tools and the police-station.
- Investing early ticks in boots or a pickaxe to compound savings over the rest of the run.
- Re-planning routes as tools change the effective weights of the map.

Level 4

Upkeep is introduced on the widest map. In addition to previous levels, focus on:

- Deciding when spending 5 ticks on an upkeep boost pays back in extra Enteloot production.
- Choosing which towns to boost, and timing boosts over high-value production windows.
- Weighing upkeep-boost ticks against construction and trade for the final score.

Scoring

All scoring inputs are taken from the state at the final tick, except invested Enteloot, which accumulates during the run.

Level 1

Contribution to the score is your generation of Enteloot, as well as the value of the items you hold at the end. A multiplier will be applied based on the number of items you sold.

Level 2 & Level 3

Infrastructure becomes the primary driver. Development spread across towns earns a multiplier.

Level 4

Upkeep activity is unlocked. Upkeep does not add a dedicated scoring term; it affects the base score only **indirectly**: the 5 ticks spent triggering a boost raise a town's Enteloot production (and thus overall Enteloot), but those ticks are unavailable for travel, gathering, crafting, or trading elsewhere.

Submissions

Participants must submit one solution per level on the Entelect Hackathon website. Each submission must include:

- A ZIP containing the source code used for that level
- A .txt file containing the JSON output for that level

The solution must be deterministic: given the same input, the program must always produce the same output. During validation, the submitted source code will be executed to reproduce the results provided in the submission. If the source code does not produce the same output as the submitted file, the submission will be considered invalid.

The .txt file must contain a valid JSON object describing the ordered list of actions for the run:

```
{
  "actions": [
    { "type": "travel", "destination": "N1" },
    { "type": "gather" },
    { "type": "gather" },
    { "type": "travel", "destination": "Demacia" },
    { "type": "craft", "item": "bread", "quantity": 3 },
    { "type": "travel", "destination": "Piltover" },
    { "type": "sell", "item": "bread", "quantity": 3 },
    { "type": "buy", "item": "fish", "quantity": 4 },
    { "type": "craft", "item": "fish-n-chips", "quantity": 2 },
    { "type": "sell", "item": "fish-n-chips", "quantity": 2 },
    { "type": "build", "upgrade": "farmhouse" }
  ]
}
```

Action fields:

Action type	Required fields
travel	destination, fast (optional, default false)
gather	none (uses the current node's activity)
buy	item, quantity
sell	item, quantity
craft	item, quantity
build	upgrade
upkeep	none (boosts the current town)

Submission validity: The submission is rejected outright with a score of 0, and nothing is simulated, if the `.txt` file is not valid JSON or if the top-level `actions` key is missing or is not an array. Once a well-formed `actions` array exists, every entry in it is handled per Assumption 4: An entry missing a required field for its `type`, with a field of the wrong type, or with an unrecognized `type`, is simply an invalid action - skipped, 1 tick consumed, logged, and the run continues.

The engine returns a per-action log showing ticks consumed, resources and Enteloot gained or spent, and running totals, so you can analyse your inputs against the outcome received.

Worked Example

Starting at Demacia (wheat trickle 2 per 10 ticks) with 200 Enteloot. Node N1 (Fields, yield 6) is 2 ticks away. Piltover pays 40 per bread and is 3 ticks from Demacia.

#	Action	Ticks	Running tick	Enteloot	Inventory
1	travel N1	2	2	200	0
2	gather	2	4	200	6 wheat
3	gather	2	6	200	12 wheat
4	travel Demacia	2	8	200	12 wheat
5	craft bread ×4	4	12	200	4 bread (1 tick per item with Demacia's crafting affinity; 8 ticks anywhere else)
6	travel Piltover	3	15	200	4 bread
7	sell bread ×4	1	16	360	0

Sixteen ticks turn 12 gathered wheat into 160 Enteloot of profit. Selling the raw wheat instead would have earned 24. Crafting and hauling multiplied the return by more than six times, and crafting at the affinity town rather than at Piltover saved 4 ticks.

Appendix

Objects

Run:

Property Name	JSON Property Name	Unit	Explanation
Total Ticks	total_ticks	Ticks	The length of the run. Scoring is done on the state at the final tick
Starting Town	starting_town	N/A	The town where the player begins
Starting Enteloot	starting_enteloot	Enteloot	The player's Enteloot at tick 0

Town:

Property Name	JSON Property Name	Unit	Explanation
Production Rate	production.rate	Ticks	Ticks per production cycle
Production Resources	production.resources	Units	Resources and amounts produced per production cycle
Upgrades	upgrades	N/A	Upgrades already built in the town at the start of the run
Affinities	affinities	N/A	Currently only crafting. Crafting reduces craft time at this town from 2 ticks per item to 1
Item Rates	item-rates	Enteloot	The price this town pays per unit of each crafted good
Enteloot Rate	enteloot.rate	Ticks	Ticks per Enteloot generation cycle
Enteloot Amount	enteloot.amount	Enteloot	Enteloot generated each generation cycle

Node:

Property Name	JSON Property Name	Unit	Explanation
Node Type	type	N/A	Fields, forest, quarry, clay-pit, fishing-grounds, pasture, or mine
Resource	resource	N/A	The resource this node yields
Yield	yield	Units	Units received per gather action
Gather Time	gather-time	Ticks	Ticks per gather action

Route:

Property Name	JSON Property Name	Unit	Explanation
Endpoints	between	N/A	The two vertices this route connects
Weight	weight	Ticks	Travel time
Toll	toll	Enteloot	Fixed cost charged on entry. 0 for standard routes

Recipe:

Property Name	JSON Property Name	Unit	Explanation
Inputs	inputs	Units	Resources (or components) consumed per craft
Craft Time	craft_time	Ticks	Ticks per craft action
Sellable	sellable	N/A	True for goods, false for construction components

Upgrade:

Property Name	JSON Property Name	Unit	Explanation
Components	components	Units	Components consumed by the Build action
Enteloot Cost	enteloot_cost	Enteloot	Enteloot consumed by the Build action
Build Time	build_time	Ticks	Ticks the Build action takes
Prerequisite	prerequisite	N/A	Condition that must hold at this town before building
Effect	effect	N/A	The buff applied to the town
Score Value	score_value	Points	Contribution to infrastructure_score

Tool:

Property Name	JSON Property Name	Unit	Explanation
Inputs	inputs	Units	Components consumed when crafting the tool
Effect	effect	N/A	The permanent buff applied for the rest of the run
Once Per Run	once_per_run	N/A	Always true. Each tool can be crafted at most once

JSON level file example

```
{
  "run": {
    "total_ticks": 500,
    "starting_town": "Demacia",
    "starting_enteloot": 200
  },
  "towns": {
    "Demacia": {
      "production": { "rate": 10, "resources": { "wheat": 2, "sheep": 2 } },
      "upgrades": [],
      "affinities": ["crafting"],
      "item-rates": {
        "bread": 10, "pottery": 50, "fish-n-chips": 30,
        "wooden-crafts": 40, "stone-works": 60, "wool-garments": 30,
        "stew": 30, "furniture": 50, "roof-tiles": 55
      },
      "enteloot": { "rate": 5, "amount": 1000 }
    },
    "Noxus": {
      "production": { "rate": 10, "resources": { "stone": 2, "wood": 2 } },
      "upgrades": [],
      "affinities": [],
      "item-rates": {
        "bread": 40, "pottery": 50, "fish-n-chips": 50,
        "wooden-crafts": 20, "stone-works": 40, "wool-garments": 40,
        "stew": 50, "furniture": 35, "roof-tiles": 40
      },
      "enteloot": { "rate": 7, "amount": 2000 }
    },
    "Piltover": {
      "production": { "rate": 10, "resources": { "fish": 2, "sheep": 2 } },
      "upgrades": [],
      "affinities": ["crafting"],
      "item-rates": {
        "bread": 40, "pottery": 70, "fish-n-chips": 10,
        "wooden-crafts": 50, "stone-works": 60, "wool-garments": 60,
        "stew": 25, "furniture": 65, "roof-tiles": 70
      },
      "enteloot": { "rate": 2, "amount": 400 }
    }
  },
  "nodes": {
    "N1": { "type": "fields", "resource": "wheat", "yield": 6, "gather-time": 2 },
    "N2": { "type": "forest", "resource": "wood", "yield": 5, "gather-time": 2 },
    "N3": { "type": "fishing-grounds", "resource": "fish", "yield": 4, "gather-time": 2 }
  },
  "routes": [
    { "between": ["Demacia", "N1"], "weight": 2, "toll": 0 },
    { "between": ["N1", "N2"], "weight": 1, "toll": 0 },
    { "between": ["N2", "Noxus"], "weight": 3, "toll": 0 },
    { "between": ["Demacia", "Piltover"], "weight": 5, "toll": 0 },
    { "between": ["Demacia", "Piltover"], "weight": 2, "toll": 50 },
    { "between": ["Piltover", "N3"], "weight": 2, "toll": 0 },
    { "between": ["Demacia", "Noxus"], "weight": 4, "toll": 0 }
  ]
}
```

Terms

Term	Explanation
Tick	The unit of simulation time. Every action consumes a whole number of ticks
Node	A map vertex where a gathering activity can be performed
Component	A crafted item consumed by Building actions, never sold
Toll	A fixed Enteloot cost charged on entering a fast route

Calculations & Algorithms

Accumulated trickle at any tick:

```
accumulated = floor(current_tick / rate) × amount
```

Ticks to fulfil a recipe from a node trip, useful for route planning:

```
trip_ticks = travel_to_node + ceil(inputs_needed / node_yield) × gather_time
             + travel_back + quantity × craft_time
```

where `craft_time` is 2, or 1 at a town with crafting affinity.

Penalties

You will incur penalties for the following:

- **Invalid actions** are skipped and consume 1 tick. Time is the scarcest resource; wasted ticks are the penalty.
- **Fast route tolls** are charged on entry. Attempting a fast route without sufficient Enteloot is an invalid action.

Ideas

Transportation Buildings

Upgrade	Effect	Components	Enteloot	Build time	Prerequisite	Score value
Stable	Unlocks **Horse** travel. Travel time **−2** (minimum 2 ticks).	6 planks + 2 bricks + 2 rope	900	4 ticks	Any 1 Civic upgrade	2,500
Port	Unlocks **Port** travel. Travel time **−5** (minimum 2 ticks).	8 planks + 6 bricks + 2 iron + 2 rope + 1 Net	1,600	6 ticks	Stable	5,000
Railway Station	Unlocks **Train** travel. Travel	12 bricks + 8 Stone-blocks + 6 Iron-	3,000	8 ticks	Pier	8,500

	time **−8** (minimum 2 ticks).	fittings + 4 Kiln-glass				

Unlimited Upgrades

Possibly allow unlimited upgrades per town instead of just one of each per town.

Other distribution multiplier equation

```
distribution_multiplier = number_of_upgrades/total_number_of_possible_upgrades
```

Bonus for Upkeep/Boosting

Add a bonus to level 4 to encourage boosting so that it is mot ignored entirely.

Research affinity

Re-look at what research affinity is, and what it does. (A research affinity reduces Research resource costs at that town).